

Track C

HealthCare - AI-Based Healthcare Navigation and Patient Assistance System

1. Problem Statement

Millions of people, particularly in rural and underserved regions, face significant challenges in accessing timely and appropriate healthcare due to limited medical facilities, low health literacy, and fragmented healthcare services. Patients often struggle to assess the urgency of their symptoms, identify the appropriate medical specialist, locate nearby healthcare facilities, and understand their eligibility for government healthcare schemes. Existing healthcare applications often provide these services independently, requiring users to switch between multiple platforms to obtain complete healthcare assistance.

Many existing AI-based healthcare systems primarily rely on single-shot symptom inputs and may not collect sufficient information through interactive follow-up questions. They may also provide recommendations without clear reasoning or source-based evidence, particularly when presenting information about government healthcare schemes. Additionally, healthcare scheme eligibility often depends on multiple criteria such as age, income, state, and beneficiary-related conditions that may be distributed across different official documents. This makes it difficult for users to obtain reliable, transparent, and understandable eligibility information.

These limitations highlight the need for an integrated healthcare navigation and patient assistance system that can conduct conversational symptom intake, provide transparent urgency-based routing, recommend appropriate medical specialists and nearby healthcare facilities, and assist users in determining their eligibility for government healthcare schemes using information retrieved from official documents. The system aims to improve healthcare navigation while providing evidence-grounded and traceable assistance for government healthcare scheme eligibility.

2. Proposed Solution

The proposed solution is an **AI-Based Healthcare Navigation and Patient Assistance System** that integrates healthcare navigation and government healthcare scheme assistance into a unified platform. The system uses **Gemini 2.5 Flash** to provide conversational symptom intake, where the user interacts with the system through multiple turns and receives relevant follow-up questions to collect complete and structured symptom information rather than relying on a single-shot symptom input.

The collected symptom information is processed by a **Python-based rule engine** to perform transparent severity and urgency classification using predefined clinical red-flag criteria grounded in relevant medical literature. Based on the identified symptom cluster and urgency level, the system recommends an appropriate medical specialist. The system then uses the **Google Maps API** to identify nearby hospitals or healthcare facilities relevant to the recommended specialist and the user's location.

The primary research contribution of the system is a **Retrieval-Augmented Generation (RAG) based government healthcare scheme eligibility framework**. Official **PM-JAY and state government healthcare scheme documents** are processed, divided into relevant sections, converted into vector embeddings using **Sentence Transformers**, and stored in **ChromaDB** for semantic retrieval. When a user requests scheme assistance, the system decomposes the eligibility requirements into relevant criteria such as age, income, state, beneficiary information, and applicable health or condition-related requirements. Relevant evidence is retrieved from multiple official scheme documents and filtered before being used for eligibility reasoning.

A dedicated **eligibility decision module** compares the user's available information against the retrieved eligibility criteria. The system can identify whether individual criteria are satisfied, not satisfied, or require additional information. The final result can therefore be classified as **Eligible, Not Eligible, Possibly Eligible, or Insufficient Information** rather than forcing a binary decision. Each eligibility claim is supported by retrieved evidence, including the relevant scheme document and page or source information, making the response traceable and source-grounded.

The system also provides **JWT-based authentication, patient profile management, patient context management, consultation history, a unified patient dashboard, and basic healthcare safety tips**. An optional document-upload facility may be included when a particular government healthcare scheme requires supporting documentation, such as an income or eligibility certificate. The system is implemented using **Next.js and Tailwind CSS** for the frontend, **FastAPI with PostgreSQL and SQLAlchemy** for the backend and persistent data management, and **LangChain** for the RAG and relevant LLM workflows.

The overall system therefore combines **conversational healthcare navigation with evidence-grounded government healthcare scheme eligibility reasoning**, with particular emphasis on multi-document retrieval, criterion-level eligibility evaluation, and source attribution.

MVP Features

1. **User Authentication** – Secure user registration and login using JWT-based authentication.
2. **Patient Profile Management** – Manage patient information including age, gender, medical history, allergies, medications, existing health conditions, and emergency contact details.
3. **Conversational Symptom Intake** – Collect comprehensive symptom information through multi-turn conversations and relevant follow-up questions instead of relying on a single-shot symptom input.
4. **Severity & Urgency Classification** – Classify symptom severity and urgency using a transparent, rule-based engine grounded in relevant clinical literature and predefined medical red-flag criteria.
5. **Specialist Recommendation** – Recommend an appropriate medical specialist based on the identified symptom cluster and severity or urgency level.
6. **Nearby Hospital Finder** – Identify and recommend nearby hospitals or healthcare facilities using the Google Maps API based on the user's location and recommended specialist or healthcare need.
7. **Government Healthcare Scheme Assistance** – Provide government healthcare scheme information and eligibility assistance using a Retrieval-Augmented Generation (RAG) pipeline based on official PM-JAY and state healthcare scheme documents.
8. **Eligibility Query Decomposition** – Break down scheme eligibility questions into individual criteria such as age, income, state, beneficiary information, and applicable health or condition-related requirements to improve targeted retrieval and reasoning.
9. **Multi-Document Eligibility Reasoning** – Retrieve and cross-reference eligibility criteria from multiple official government scheme documents and evaluate the user's information against the relevant requirements.
10. **Evidence Filtering and Eligibility Decision** – Filter retrieved evidence and use a structured eligibility decision process to determine whether individual eligibility criteria are satisfied, not satisfied, or require additional information.
11. **Source-Grounded Eligibility Answers** – Provide eligibility results with supporting evidence, including the relevant scheme, document, page number, or retrieved source used for each eligibility claim.

12. **Consultation & Recommendation History** – Maintain records of previous consultations, collected symptoms, severity assessments, specialist recommendations, hospital recommendations, and government healthcare scheme queries and results.
13. **Patient Context Management** – Maintain relevant patient profile information, medical history, allergies, medications, and previous consultation information to provide context-aware assistance during subsequent consultations.
14. **Patient Dashboard** – Provide a unified dashboard for accessing the patient profile, consultation history, healthcare recommendations, hospital information, and government healthcare scheme results.
15. **Basic Healthcare Tips** – Provide simple, general healthcare and safety tips generated using the LLM while avoiding definitive diagnosis or unsafe treatment recommendations.
16. **Optional Document Upload** – Support medical or eligibility-document uploads when required by a particular government healthcare scheme, such as an income or eligibility certificate.

Tech Stacks

Layer	Technology	Purpose
Frontend	Next.js	Develops the responsive and interactive web application for users.
UI Framework	Tailwind CSS	Builds a modern, responsive, and customizable user interface.
Backend	FastAPI	Handles API development and communication between the frontend, AI services, external APIs, and database.
Large Language Model (LLM)	Gemini 2.5 Flash	Performs conversational reasoning, follow-up questioning, structured symptom extraction, context handling, and response generation.
AI/RAG Framework	LangChain	Manages LLM workflows and builds the Retrieval-Augmented Generation pipeline for government healthcare scheme assistance.

Layer	Technology	Purpose
Severity & Urgency Logic	Python Rule Engine	Performs transparent, rule-based severity and urgency classification using predefined clinical red-flag criteria grounded in relevant medical literature.
Eligibility Decision Logic	Python Decision Engine	Compares retrieved eligibility criteria with patient information and determines criterion-level eligibility, missing information, and the overall eligibility status.
Embedding Model	Sentence Transformers	Converts official government healthcare scheme documents into vector embeddings for semantic retrieval.
Vector Database	ChromaDB	Stores and retrieves document embeddings and associated metadata for semantic search across government healthcare scheme documents.
Location Services	Google Maps API	Identifies nearby hospitals and healthcare facilities based on the user's location and recommended healthcare specialty or need.
Database	PostgreSQL	Stores user accounts, patient profiles, consultations, conversations, symptoms, assessments, recommendations, scheme queries, eligibility results, and supporting evidence.
ORM	SQLAlchemy	Handles database operations and relationships between the FastAPI backend and PostgreSQL.
Authentication	JWT	Provides secure user authentication and authorization.
File Storage	Cloudinary (Optional)	Stores uploaded eligibility or medical documents when document upload is required by a government healthcare scheme.

Workflow

1. **User registers or logs into the system** using JWT-based authentication.

2. **The user creates or updates their patient profile** with relevant information such as age, gender, medical history, allergies, medications, existing health conditions, and emergency contact details.
3. **The system retrieves relevant patient context** from the patient's profile and previous consultation history to support the current interaction.
4. **The user enters their symptoms through a conversational text interface.**
5. **Gemini 2.5 Flash performs conversational symptom intake** by analyzing the initial user input and asking relevant follow-up questions to collect complete symptom information.
6. **The system converts the conversation into structured symptom information**, including the primary symptom, associated symptoms, duration, onset, severity indicators, and relevant risk factors.
7. **The structured symptom information is passed to the Python-based Severity and Urgency Rule Engine**, which evaluates predefined clinical red-flag criteria grounded in relevant medical literature.
8. **The system determines the urgency level** of the user's symptoms, such as emergency, urgent, non-urgent, or routine, and records the rules or red flags that contributed to the classification.
9. **The Specialist Recommendation module** maps the identified symptom cluster and urgency level to an appropriate medical specialist. If a specific specialty cannot be determined, the system may recommend a general physician.
10. **The Google Maps API is used to identify nearby healthcare facilities** based on the user's location and the recommended specialist or healthcare need.
11. **When the user requests government healthcare scheme assistance**, the system collects the relevant patient information required for eligibility evaluation.
12. **The eligibility query is decomposed into individual criteria**, such as age, income, state, beneficiary information, and applicable health or condition-related requirements.
13. **The RAG pipeline retrieves relevant information from official PM-JAY and state government healthcare scheme documents** using Sentence Transformers for embeddings and ChromaDB for semantic retrieval.
14. **Retrieved evidence is filtered and organized according to the individual eligibility criteria.** Relevant information from multiple official documents can be

combined when eligibility requirements are distributed across different documents.

15. **The Eligibility Decision Engine compares the patient's information against the retrieved eligibility criteria** and determines whether each criterion is satisfied, not satisfied, or requires additional information.
16. **The system generates a source-grounded eligibility response** with an overall status such as Eligible, Not Eligible, Possibly Eligible, or Insufficient Information. Each important eligibility claim is associated with supporting evidence and relevant document or page information.
17. **The system stores consultation and recommendation information in PostgreSQL**, including conversation messages, structured symptoms, patient context, severity assessments, specialist recommendations, hospital recommendations, scheme queries, eligibility results, and supporting evidence.
18. **The patient can access the stored information through the dashboard and consultation history**, allowing previous assessments, recommendations, and government scheme results to be reviewed.
19. **Basic healthcare tips may be generated using Gemini 2.5 Flash** for appropriate situations. These tips are limited to general healthcare and safety guidance and do not provide definitive diagnosis or treatment.
20. **Optional document upload may be used when a government scheme requires supporting documentation**, such as an income or eligibility certificate. Such documents can be stored using Cloudinary when the optional upload feature is implemented.

Person 1 – Frontend Developer

Responsibilities

- UI/UX Design
- Next.js Application Development
- Tailwind CSS
- API Integration
- Form Validation
- State Management
- Responsive Design

- Patient Dashboard Development
 - Patient Context and Consultation History Interface
 - Healthcare Scheme Eligibility Interface
-

Modules

1. Authentication UI

- Login
- Registration
- Forgot Password
- JWT token handling
- Authentication state management

2. Patient Dashboard

- Home page
- Navigation
- Patient summary
- Recent consultations
- Latest severity and urgency assessment
- Specialist recommendation summary
- Nearby hospital recommendations
- Government healthcare scheme summary
- Quick access to major healthcare services
- Sidebar/Navbar

3. Patient Profile

- Personal details
- Age and gender
- Medical history
- Allergies
- Medications

- Existing health conditions
- Emergency contact information

4. Symptom Assessment Interface

- Text-based symptom input
- Chat-like conversational interface
- Display Gemini-generated follow-up questions
- Display patient's previous responses
- Typing/loading animation
- Submit symptoms for assessment
- Display structured symptom information
- Indicate completion of symptom intake

5. Severity & Urgency Recommendation

- Display urgency level
- Emergency/urgent/non-urgent/routine status
- Display triggered red flags where appropriate
- Display severity/urgency explanation
- Display recommended action

6. Specialist Recommendation

- Display recommended medical specialist
- Display reason for recommendation
- Display urgency-related recommendation
- Provide navigation to hospital search

7. Hospital Recommendation Page

- Google Maps integration
- Nearby hospital/healthcare facility cards
- Hospital name and address
- Distance from user's location
- Healthcare facility details

- Directions/location navigation
- Specialist-related search information

8. Government Healthcare Scheme Page

- Scheme search/query interface
- Display relevant government schemes
- Display scheme description and benefits
- Display eligibility criteria
- Display eligibility status
- Display criterion-level results
- Display missing information when required
- Display supporting evidence
- Display source document and page information

9. Consultation & Recommendation History

- Previous consultations
- Previous symptoms
- Structured symptom information
- Severity and urgency assessments
- Specialist recommendations
- Hospital recommendations
- Government scheme queries
- Eligibility results
- Consultation date and time

10. Patient Context View

- Patient medical history summary
- Relevant allergies and medications
- Existing health conditions
- Relevant previous consultations
- Patient information used during the current assessment

- Context-aware information display

11. Optional Document Upload

- Upload eligibility-related documents
- Display uploaded documents
- Display document type and upload date
- View/download documents
- Delete documents
- Display document-processing status where applicable

12. Basic Healthcare Tips

- Display general healthcare and safety tips
- Display appropriate escalation guidance
- Clearly distinguish general guidance from professional medical care

13. Responsive Design

- Desktop interface
- Tablet interface
- Mobile-responsive layouts
- Accessible forms and navigation
- Consistent UI components across all modules

Person 2 – Backend + AI Developer

Responsibilities

- FastAPI Backend Development
- PostgreSQL Database Management
- SQLAlchemy ORM
- JWT Authentication
- Gemini 2.5 Flash Integration
- LangChain
- Retrieval-Augmented Generation (RAG)

- Python Rule Engine
 - Eligibility Decision Engine
 - Google Maps API Integration
 - Patient Context Management
 - Consultation and History Management
 - Optional File Storage
 - API Development
 - Business Logic
-

Modules

1. Authentication API

- User registration
- User login
- JWT authentication
- Token refresh
- Password management
- Authentication and authorization

2. Patient Management API

- CRUD operations for patient profiles
- Patient demographic information
- Medical history
- Allergies
- Medications
- Existing health conditions
- Emergency contacts

3. Conversational Symptom Intake

- Gemini 2.5 Flash integration
- Multi-turn conversation management

- Follow-up question generation
- Identification of missing symptom information
- Structured symptom extraction
- Patient context integration
- Conversation state management

4. Patient Context Management

- Retrieve relevant patient profile information
- Retrieve medical history, allergies, medications, and existing conditions
- Retrieve relevant previous consultations
- Build consultation-specific patient context
- Provide relevant context to the conversational symptom intake module

5. Structured Symptom Assessment

- Process conversational responses
- Extract structured symptom information
- Store primary and associated symptoms
- Store symptom duration and onset
- Store severity indicators
- Store relevant risk factors
- Prepare structured information for the severity and urgency engine

6. Severity & Urgency Assessment

- Python-based rule engine
- Clinical red-flag evaluation
- Emergency classification
- Urgent classification
- Non-urgent classification
- Routine classification
- Triggered-rule identification
- Severity and urgency explanation

- Literature-grounded rule management

7. Specialist Recommendation

- Symptom-cluster-to-specialist mapping
- Urgency-based recommendation
- General physician fallback when a specific specialty cannot be determined
- Recommendation reasoning
- Specialist recommendation API

8. Hospital Recommendation

- Google Maps API integration
- Nearby hospital/healthcare facility search
- Location-based recommendations
- Specialist-related search
- Distance calculation
- Hospital details retrieval
- Consultation-specific hospital recommendations

9. Government Healthcare Scheme RAG

- Collection of official PM-JAY and state government healthcare scheme documents
- PDF/document text extraction
- Document cleaning and chunking
- Sentence Transformer embeddings
- ChromaDB vector storage
- Semantic retrieval
- LangChain RAG workflow
- Gemini 2.5 Flash response generation
- Retrieval of eligibility criteria
- Retrieval of scheme benefits
- Multi-document information retrieval

10. Eligibility Query Decomposition

- Analyze the user's scheme-related question
- Identify eligibility criteria required for evaluation
- Decompose the query into criteria such as:
 - Age
 - Income
 - State/residence
 - Beneficiary/category information
 - Applicable health or condition-related requirements
 - Required supporting documents
- Generate targeted retrieval queries for each criterion

11. Evidence Filtering & Retrieval

- Retrieve relevant document chunks from ChromaDB
- Filter irrelevant retrieved content
- Associate retrieved evidence with individual eligibility criteria
- Maintain document metadata
- Track source document and page information
- Prepare relevant evidence for eligibility reasoning

12. Multi-Document Eligibility Reasoning

- Combine eligibility information retrieved from multiple official documents
- Cross-reference scheme-specific criteria
- Compare patient information against retrieved requirements
- Evaluate individual eligibility criteria
- Identify conflicting or incomplete information
- Identify missing information required for eligibility assessment

13. Eligibility Decision Engine

- Compare structured patient information with retrieved eligibility criteria
- Determine criterion-level results:

- PASS
- FAIL
- UNKNOWN
- Determine overall eligibility status:
 - ELIGIBLE
 - NOT_ELIGIBLE
 - POSSIBLY_ELIGIBLE
 - INSUFFICIENT_INFORMATION
- Generate the reasoning behind the eligibility result

14. Source-Grounded Eligibility Response

- Link eligibility claims to retrieved evidence
- Store supporting document information
- Store page numbers and chunk references
- Provide source-grounded responses
- Prevent unsupported eligibility claims
- Return evidence supporting each important eligibility criterion

15. Optional Document Upload

- Eligibility-document upload
- Document validation
- Cloudinary storage
- Document metadata management
- Document retrieval
- Support documents such as income or eligibility certificates when required by a scheme

16. Consultation & History Management

- Store consultation sessions
- Store conversation messages
- Store structured symptoms

- Store patient context
- Store severity and urgency assessments
- Store specialist recommendations
- Store hospital recommendations
- Store government scheme queries
- Store eligibility results
- Store supporting eligibility evidence
- Retrieve previous consultation information

17. Patient Dashboard API

- Patient summary
- Recent consultations
- Latest severity and urgency assessment
- Specialist recommendations
- Hospital recommendations
- Government healthcare scheme information
- Eligibility results
- Consultation history

18. Basic Healthcare Tips

- Generate general healthcare and safety guidance using Gemini 2.5 Flash
- Provide appropriate escalation guidance
- Restrict output to general healthcare information
- Avoid definitive diagnosis and treatment recommendations

19. Database

- PostgreSQL
- SQLAlchemy
- Database models
- CRUD operations
- Data relationships

- Consultation and history management
- Scheme and document metadata management
- Eligibility evidence storage
- Patient context storage

20. Activity Logging

- Record important user activities
- Track authentication and major healthcare-system interactions
- Store activity timestamps
- Support basic system auditing

8. Database Schema

The system uses **PostgreSQL** as the primary relational database, with **SQLAlchemy** as the ORM. The database stores authentication information, patient profiles, conversational consultations, structured symptoms, severity assessments, specialist and hospital recommendations, government healthcare scheme queries, eligibility results, and supporting evidence.

1. users

Purpose: Stores user account and authentication information.

Column	Type	Constraints
user_id	UUID	Primary Key
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL
phone	VARCHAR(15)	UNIQUE
role	VARCHAR(20)	DEFAULT 'PATIENT'
is_verified	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	

Column	Type	Constraints
updated_at	TIMESTAMP	

2. patient_profiles

Purpose: Stores patient demographic and profile information.

Column	Type	Constraints
profile_id	UUID	Primary Key
user_id	UUID	FK → users.user_id, UNIQUE
date_of_birth	DATE	
gender	VARCHAR(20)	
blood_group	VARCHAR(10)	
height_cm	NUMERIC(5,2)	
weight_kg	NUMERIC(5,2)	
address	TEXT	
city	VARCHAR(100)	
state	VARCHAR(100)	
pincode	VARCHAR(10)	
emergency_contact_name	VARCHAR(100)	
emergency_contact_phone	VARCHAR(15)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

3. allergies

Purpose: Stores allergies associated with a patient.

Column	Type	Constraints
allergy_id	UUID	Primary Key
profile_id	UUID	FK → patient_profiles.profile_id
allergy_name	VARCHAR(100) NOT NULL	
severity	VARCHAR(30)	
notes	TEXT	

4. chronic_conditions

Purpose: Stores existing medical conditions of a patient.

Column	Type	Constraints
condition_id	UUID	Primary Key
profile_id	UUID	FK → patient_profiles.profile_id
condition_name	VARCHAR(150) NOT NULL	
diagnosed_year	INTEGER	
notes	TEXT	

5. medications

Purpose: Stores medications currently used by the patient.

Column	Type	Constraints
medication_id	UUID	Primary Key
profile_id	UUID	FK → patient_profiles.profile_id
medicine_name	VARCHAR(150) NOT NULL	
dosage	VARCHAR(50)	
frequency	VARCHAR(100)	
prescribed_by	VARCHAR(100)	

Consultation & Symptom Assessment

6. conversations

Purpose: Stores each conversational symptom-assessment session.

Column	Type	Constraints
conversation_id	UUID	Primary Key
profile_id	UUID	FK → patient_profiles.profile_id
started_at	TIMESTAMP	
ended_at	TIMESTAMP	
status	VARCHAR(30)	

Possible values:

ACTIVE

COMPLETED

CANCELLED

7. conversation_messages

Purpose: Stores user messages, follow-up questions, and AI responses during a consultation.

Column	Type	Constraints
message_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id
sender	VARCHAR(20) NOT NULL	
message	TEXT	NOT NULL
created_at	TIMESTAMP	

Possible sender values:

USER

ASSISTANT

SYSTEM

8. symptom_assessments

Purpose: Stores the structured symptom information extracted from the conversational assessment.

Column	Type	Constraints
assessment_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id
primary_symptom	VARCHAR(150)	
symptoms	JSONB	
duration	VARCHAR(100)	
onset	VARCHAR(100)	
severity_description	VARCHAR(100)	
associated_symptoms	JSONB	
risk_factors	JSONB	
structured_data	JSONB	
created_at	TIMESTAMP	

The JSONB fields allow the system to store different symptom characteristics without creating separate database columns for every possible symptom.

9. patient_context

Purpose: Stores relevant patient information considered during a particular consultation.

Column	Type	Constraints
context_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id

Column	Type	Constraints
profile_id	UUID	FK → patient_profiles.profile_id
relevant_history	TEXT	
relevant_conditions	JSONB	
relevant_allergies	JSONB	
relevant_medications	JSONB	
previous_consultations	JSONB	
created_at	TIMESTAMP	

Severity & Specialist Recommendation

10. severity_assessments

Purpose: Stores the results generated by the Python-based severity and urgency rule engine.

Column	Type	Constraints
assessment_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id
severity_level	VARCHAR(30)	NOT NULL
urgency_level	VARCHAR(30)	NOT NULL
emergency_flag	BOOLEAN	DEFAULT FALSE
triggered_rules	JSONB	
explanation	TEXT	
assessed_at	TIMESTAMP	

Possible urgency levels:

ROUTINE

NON_URGENT

URGENT

EMERGENCY

The triggered_rules field stores the clinical red flags or predefined rules that contributed to the assessment.

11. specialist_recommendations

Purpose: Stores the recommended medical specialist for a consultation.

Column	Type	Constraints
recommendation_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id
specialist	VARCHAR(150)	NOT NULL
reason	TEXT	
recommended_at	TIMESTAMP	

Hospital Recommendation

12. hospitals

Purpose: Stores healthcare facility information retrieved through Google Maps/Places services.

Column	Type	Constraints
hospital_id	UUID	Primary Key
google_place_id	VARCHAR(255)	UNIQUE
hospital_name	VARCHAR(200)	NOT NULL
address	TEXT	
city	VARCHAR(100)	
state	VARCHAR(100)	
latitude	DECIMAL(10,7)	
longitude	DECIMAL(10,7)	

Column	Type	Constraints
phone	VARCHAR(20)	
website	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

13. hospital_recommendations

Purpose: Stores healthcare facilities recommended for a particular consultation.

Column	Type	Constraints
recommendation_id	UUID	Primary Key
conversation_id	UUID	FK → conversations.conversation_id
hospital_id	UUID	FK → hospitals.hospital_id
specialist_type	VARCHAR(150)	
distance_km	DECIMAL(8,2)	
estimated_time_minutes	INTEGER	
recommended_at	TIMESTAMP	

Government Healthcare Scheme Module

14. government_schemes

Purpose: Stores metadata about government healthcare schemes.

Column	Type	Constraints
scheme_id	UUID	Primary Key
scheme_name	VARCHAR(250)	NOT NULL
department	VARCHAR(200)	
description	TEXT	

Column	Type	Constraints
official_url	TEXT	
state	VARCHAR(100)	
last_updated	DATE	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

15. scheme_documents

Purpose: Stores metadata about official government documents used by the RAG pipeline.

Column	Type	Constraints
document_id	UUID	Primary Key
scheme_id	UUID	FK → government_schemes.scheme_id
document_name	VARCHAR(255)	NOT NULL
document_type	VARCHAR(100)	
official_url	TEXT	
version	VARCHAR(50)	
published_date	DATE	
last_updated	DATE	
created_at	TIMESTAMP	

These documents may include PM-JAY guidelines, state scheme guidelines, eligibility documents, and scheme benefit documents.

16. scheme_queries

Purpose: Stores user queries and the resulting government healthcare scheme eligibility assessments.

Column	Type	Constraints
query_id	UUID	Primary Key
profile_id	UUID	FK → patient_profiles.profile_id
conversation_id	UUID	FK → conversations.conversation_id, NULL
user_question	TEXT	NOT NULL
ai_response	TEXT	
eligibility_status	VARCHAR(50)	
missing_information	JSONB	
created_at	TIMESTAMP	

Possible eligibility statuses:

ELIGIBLE

NOT_ELIGIBLE

POSSIBLY_ELIGIBLE

INSUFFICIENT_INFORMATION

17. eligibility_evidence

Purpose: Stores the evidence supporting individual eligibility claims.

Column	Type	Constraints
evidence_id	UUID	Primary Key
query_id	UUID	FK → scheme_queries.query_id
document_id	UUID	FK → scheme_documents.document_id
page_number	INTEGER	
chunk_id	VARCHAR(150)	
criterion	VARCHAR(150)	
evidence_text	TEXT	

Column	Type	Constraints
--------	------	-------------

criterion_result	VARCHAR(20)	
------------------	-------------	--

created_at	TIMESTAMP	
------------	-----------	--

Possible criterion_result values:

PASS

FAIL

UNKNOWN

This table provides traceability between an eligibility decision and the official document evidence used to support that decision.

Optional Document Upload

18. uploaded_documents

Purpose: Stores documents uploaded by users when a government healthcare scheme requires supporting documentation.

Column	Type	Constraints
--------	------	-------------

document_id	UUID	Primary Key
-------------	------	-------------

profile_id	UUID	FK → patient_profiles.profile_id
------------	------	----------------------------------

scheme_query_id	UUID	FK → scheme_queries.query_id, NULL
-----------------	------	------------------------------------

file_name	VARCHAR(255)	NOT NULL
-----------	--------------	----------

file_type	VARCHAR(50)	
-----------	-------------	--

category	VARCHAR(100)	
----------	--------------	--

cloudinary_url	TEXT	
----------------	------	--

upload_date	TIMESTAMP	
-------------	-----------	--

processing_status	VARCHAR(30)	
-------------------	-------------	--

Possible categories include:

INCOME_CERTIFICATE

ELIGIBILITY_CERTIFICATE

OTHER

This table can be omitted if optional document upload is not implemented.

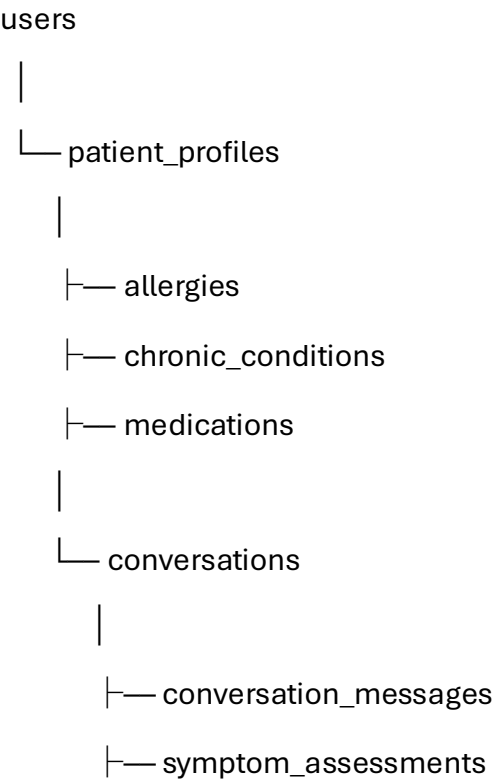
Activity Logging

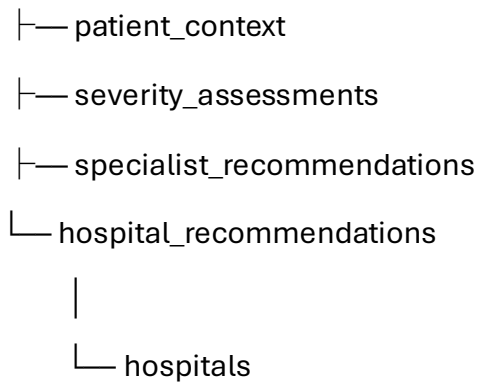
19. activity_logs

Purpose: Stores important user activities for basic system auditing.

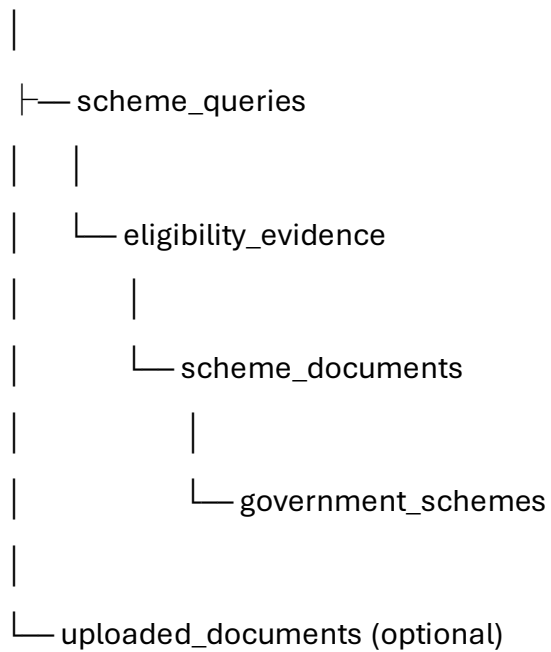
Column	Type	Constraints
activity_id	UUID	Primary Key
user_id	UUID	FK → users.user_id
activity_type	VARCHAR(100)	
description	TEXT	
ip_address	VARCHAR(50)	
created_at	TIMESTAMP	

Database Relationship Overview

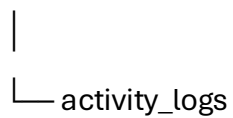




patient_profiles



users



Removed from the Previous Database

The revised schema intentionally does not contain separate tables for:

- Disease predictions
- SHAP explanations
- AI guardrail logs
- FHIR resources

- PII redaction
- Translation/voice processing

These belonged to the previous implementation and are outside the revised MVP.

REST API Endpoints

Authentication

1. POST /api/v1/auth/register
 2. POST /api/v1/auth/login
 3. POST /api/v1/auth/refresh
 4. POST /api/v1/auth/logout
 5. POST /api/v1/auth/forgot-password
 6. POST /api/v1/auth/reset-password
 7. POST /api/v1/auth/verify-email
-

User Profile

8. GET /api/v1/profile
 9. PUT /api/v1/profile
-

Allergies

10. GET /api/v1/profile/allergies
 11. POST /api/v1/profile/allergies
 12. PUT /api/v1/profile/allergies/{allergy_id}
 13. DELETE /api/v1/profile/allergies/{allergy_id}
-

Chronic Conditions

14. GET /api/v1/profile/conditions
15. POST /api/v1/profile/conditions

16. PUT /api/v1/profile/conditions/{condition_id}

17. DELETE /api/v1/profile/conditions/{condition_id}

Medications

18. GET /api/v1/profile/medications

19. POST /api/v1/profile/medications

20. PUT /api/v1/profile/medications/{medication_id}

21. DELETE /api/v1/profile/medications/{medication_id}

AI Conversation & Symptom Intake

22. POST /api/v1/conversations

23. GET /api/v1/conversations

24. GET /api/v1/conversations/{conversation_id}

25. POST /api/v1/conversations/{conversation_id}/messages

26. DELETE /api/v1/conversations/{conversation_id}

These endpoints create and manage conversational symptom-assessment sessions, including user messages and AI-generated follow-up questions.

Patient Context

27. GET /api/v1/conversations/{conversation_id}/context

28. GET /api/v1/profile/context

These endpoints retrieve relevant patient profile information, medical history, allergies, medications, existing conditions, and relevant previous consultation information used during the current assessment.

Structured Symptom Assessment

29. GET /api/v1/conversations/{conversation_id}/symptoms

30. GET /api/v1/conversations/{conversation_id}/symptoms/structured

These endpoints retrieve the structured symptom information extracted from the conversational assessment, including primary symptoms, associated symptoms, duration, onset, severity indicators, and relevant risk factors.

Severity & Urgency Assessment

31. POST /api/v1/conversations/{conversation_id}/severity

32. GET /api/v1/conversations/{conversation_id}/severity

These endpoints execute and retrieve the Python-based rule-engine assessment, including urgency level, emergency status, triggered red flags, and the explanation for the classification.

Specialist Recommendation

33. GET /api/v1/conversations/{conversation_id}/specialist

Returns the recommended medical specialist and the reason for the recommendation based on the symptom cluster and urgency level.

Hospital Recommendation

34. POST /api/v1/hospitals/nearby

35. GET /api/v1/hospitals/{hospital_id}

36. GET /api/v1/conversations/{conversation_id}/hospitals

These endpoints search for nearby hospitals or healthcare facilities using the Google Maps API and retrieve healthcare facilities recommended for a particular consultation.

Government Healthcare Schemes

37. GET /api/v1/schemes

38. GET /api/v1/schemes/{scheme_id}

39. POST /api/v1/schemes/query

40. GET /api/v1/schemes/queries

41. GET /api/v1/schemes/queries/{query_id}

These endpoints support government healthcare scheme discovery, scheme-related queries, eligibility assessment, and retrieval of previous scheme queries.

Scheme Documents & Eligibility Evidence

42. GET /api/v1/schemes/{scheme_id}/documents

43. GET /api/v1/schemes/queries/{query_id}/evidence

These endpoints retrieve official scheme-document information and the evidence used to support individual eligibility claims, including relevant document and page information.

Optional Document Upload

44. POST /api/v1/documents

45. GET /api/v1/documents

46. GET /api/v1/documents/{document_id}

47. DELETE /api/v1/documents/{document_id}

These endpoints are optional and are used when a government healthcare scheme requires supporting documentation such as an income certificate or eligibility certificate.

Consultation & Recommendation History

48. GET /api/v1/history

49. GET /api/v1/history/{conversation_id}

These endpoints retrieve previous consultations, conversation messages, structured symptoms, severity assessments, specialist recommendations, hospital recommendations, and related scheme information.

Dashboard

50. GET /api/v1/dashboard

Returns a unified patient dashboard containing the patient summary, recent consultations, latest severity and urgency assessment, specialist recommendations, hospital recommendations, and government healthcare scheme information.

Activity Logs

51. GET /api/v1/activity-logs

Retrieves the authenticated user's relevant system activity history for basic auditing.

10. Research Contribution and Research Gap

Existing healthcare AI systems have explored conversational symptom assessment, medical triage, healthcare recommendation, and Retrieval-Augmented Generation for medical information. However, these areas are generally addressed as separate components. Conversational systems such as C-PATH and PHA demonstrate the usefulness of multi-turn symptom collection and structured patient information, while systems such as AskDoc demonstrate the integration of symptom assessment, urgency classification, specialist recommendation, and nearby hospital discovery. These studies indicate that conversational healthcare navigation is feasible but do not establish a unified framework for evidence-grounded government healthcare scheme eligibility reasoning.

Healthcare RAG research has primarily focused on medical question answering, clinical information retrieval, evidence-grounded responses, and clinical decision-support applications. Existing studies demonstrate that retrieval quality, evidence selection, and grounding have a significant effect on the reliability of LLM-generated healthcare responses. However, the reviewed literature provides limited evidence of systems specifically designed to perform **multi-document eligibility reasoning over Indian government healthcare scheme documents**.

A further limitation is that conventional RAG systems generally retrieve relevant passages and generate an answer, but do not necessarily decompose an eligibility question into individual criteria and systematically evaluate each criterion against evidence collected from multiple documents. Government healthcare scheme eligibility can involve several conditions, including age, income, state, beneficiary category, health-related requirements, and supporting documentation. These criteria may be distributed across different official documents, making simple single-document retrieval insufficient for comprehensive eligibility assessment.

Therefore, the primary research contribution of the proposed system is a **multi-document, source-grounded eligibility reasoning framework for government healthcare schemes**. The framework decomposes a user's eligibility query into individual criteria, retrieves relevant evidence from official PM-JAY and state scheme

documents, filters the retrieved evidence, and evaluates each criterion against the available patient information using a structured eligibility decision process.

The system further provides **criterion-level evidence attribution**, linking important eligibility claims to the supporting scheme document and relevant page or retrieved source information. Instead of producing only a binary eligibility response, the system can identify satisfied criteria, failed criteria, and missing information and classify the overall result as **Eligible, Not Eligible, Possibly Eligible, or Insufficient Information**.

The project also integrates this eligibility framework with a conversational healthcare navigation workflow consisting of multi-turn symptom intake, literature-grounded urgency classification, specialist recommendation, and nearby healthcare facility discovery. This creates a unified patient assistance platform in which healthcare navigation and government healthcare scheme assistance are available through a common patient context.

Key Research Contributions

1. **Conversational Symptom Intake** – Uses multi-turn interaction to collect structured symptom information rather than relying on single-shot symptom input.
2. **Literature-Grounded Urgency Routing** – Uses a transparent Python-based rule engine and clinical red-flag criteria for severity and urgency classification instead of allowing the LLM to make the final urgency decision independently.
3. **Multi-Document Government Scheme RAG** – Retrieves relevant information from multiple official PM-JAY and state government healthcare scheme documents.
4. **Eligibility Query Decomposition** – Breaks complex eligibility questions into individual criteria such as age, income, state, beneficiary information, and applicable health or condition-related requirements.
5. **Evidence Filtering and Attribution** – Associates eligibility criteria with relevant retrieved evidence and identifies the supporting document and page information.
6. **Structured Eligibility Decision** – Compares patient information against retrieved eligibility requirements and determines criterion-level PASS, FAIL, or UNKNOWN results.
7. **Source-Grounded Eligibility Response** – Provides traceable eligibility explanations supported by official scheme documents rather than relying solely on the LLM's internal knowledge.

8. **Integrated Healthcare Navigation** – Combines conversational symptom assessment, urgency routing, specialist recommendation, nearby healthcare facility discovery, and government scheme assistance within a unified platform.

Research Gap Addressed

The reviewed literature indicates that existing research is relatively fragmented across conversational symptom assessment, medical triage, healthcare recommendation, and healthcare RAG. The proposed system addresses this gap by combining these capabilities while placing its primary research focus on **multi-document, evidence-grounded eligibility reasoning for Indian government healthcare schemes**.

The intended contribution is therefore not the individual use of an LLM, RAG, or conversational interface. Instead, the contribution lies in the **structured combination of query decomposition, multi-document retrieval, evidence filtering, criterion-level eligibility evaluation, and source attribution for government healthcare scheme eligibility**.